TP Chapitre 0 : Compilation et Débogage

Du Code Source à l'Exécutable

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

Environnement de travail

Ce TP doit être réalisé dans l'environnement Docker du cours. Si vous n'avez pas encore configuré l'environnement, consultez le TP-1 (Setup Environnement).

Commandes de base:

```
# Linux/macOS
docker run -it -v $(pwd):/workspace elbachirtoure/c-advanced-course:v1.0

# Windows (Git Bash)
docker run -it -v /${PWD}:/workspace elbachirtoure/c-advanced-course:v1.0

# Dans le container
cd /workspace
```

1 Phases de Compilation et Optimisations

Exercice 1 : De la source à l'exécutable

Objectifs : Observer les fichiers intermédiaires, comprendre le rôle de chaque phase, mesurer l'impact des optimisations.

Partie A – Fichiers intermédiaires. Créer `phases.c` :

```c
#include <stdio.h>

#define TAILLE 100
#define CARRE(x) ((x) * (x))

int global = 42;  // Variable globale initialisée

int somme_carres(int n) {
    int total = 0;
    for (int i = 1; i <= n; i++) {
        total += CARRE(i);
    }
    return total;
}

int main() {
    int local = TAILLE;
    printf("Somme des carrés de 1 à %d = %d\n", local, somme_carres(local));
    return 0;
}
```

Générer tous les fichiers intermédiaires :

```
gcc -E phases.c -o phases.i      # Préprocesseur
gcc -S phases.c -o phases.s      # Compilation (assembleur)
gcc -c phases.c -o phases.o      # Assemblage (objet)
gcc phases.o -o phases           # Édition des liens
```

Analyser les fichiers :

```
head -50 phases.i && tail -20 phases.i   # Fichier préprocessé
cat phases.s                              # Code assembleur
nm phases.o                               # Symboles avant linkage
nm phases                                 # Symboles après linkage
```

Partie B – Impact des optimisations. Compiler avec différents niveaux :

```
gcc -O0 -S phases.c -o phases_O0.s && gcc -O0 -o phases_O0 phases.c
gcc -O2 -S phases.c -o phases_O2.s && gcc -O2 -o phases_O2 phases.c
gcc -O3 -S phases.c -o phases_O3.s && gcc -O3 -o phases_O3 phases.c

wc -l phases_O*.s               # Comparer le nombre de lignes
ls -la phases_O*                # Comparer les tailles
diff phases_O0.s phases_O2.s | head -30
```

Questions :

1. Les macros `TAILLE` et `CARRE` apparaissent-elles dans `phases.i` ? Pourquoi ?
2. Dans `nm phases.o`, que signifient les symboles marqués `T`, `D`, et `U` ?
3. La fonction `somme_carres` est-elle présente dans l'assembleur `-O3` ? Expliquer.
4. Pourquoi le nombre de lignes assembleur peut-il augmenter avec l'optimisation ?

2 Compilation Séparée et Erreurs

Exercice 2 : Multi-fichiers et diagnostic d'erreurs

Objectifs : Maîtriser la compilation séparée, distinguer erreurs de compilation et de liaison.
Partie A – Compilation séparée. Créer trois fichiers :
`math_utils.h` :

```c
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int addition(int a, int b);
int factorielle(int n);
extern int nb_appels;
#endif
```

`math_utils.c` :

```c
#include "math_utils.h"
int nb_appels = 0;

int addition(int a, int b) { nb_appels++; return a + b; }

int factorielle(int n) {
    nb_appels++;
    return (n <= 1) ? 1 : n * factorielle(n - 1);
}
```

`main.c` :

```c
#include <stdio.h>
#include "math_utils.h"

int main() {
    printf("5 + 3 = %d\n", addition(5, 3));
    printf("5! = %d\n", factorielle(5));
    printf("Nombre d'appels: %d\n", nb_appels);
    return 0;
}
```

Compiler et analyser :

```
gcc -Wall -c math_utils.c -o math_utils.o
gcc -Wall -c main.c -o main.o
nm math_utils.o && echo "---" && nm main.o
gcc math_utils.o main.o -o programme
./programme
```

Partie B – Provoquer des erreurs. Tester ces scénarios :

```
# Erreur de compilation : oublier un point-virgule dans main.c
# Erreur de linkage : compiler sans math_utils.o
gcc main.o -o test_link

# Définition multiple : ajouter "int nb_appels = 99;" dans main.c
gcc -c main.c -o main.o   # Compile OK
gcc math_utils.o main.o -o test_multi  # Erreur ici
```

Questions :

1. Pourquoi `main.o` montre `addition` comme `U` (undefined) alors que le code compile ?
2. Quelle est la différence entre le message d'erreur du compilateur et celui du linker ?
3. Comment le linker détecte-t-il une définition multiple de `nb_appels` ?

3 Débogage avec GDB

Exercice 3 : Trouver et corriger des bugs avec GDB

Objectifs : Utiliser breakpoints, inspection de variables, exécution pas à pas, pile d'appels.
Créer `buggy.c` contenant plusieurs bugs :

```c
#include <stdio.h>

int chercher_max(int tab[], int taille) {
    int max = 0;  // Bug 1: mauvaise initialisation
    for (int i = 0; i <= taille; i++) {  // Bug 2: <= au lieu de <
        if (tab[i] > max)
            max = tab[i];
    }
    return max;
}

double moyenne(int tab[], int taille) {
    int somme = 0;
    for (int i = 0; i < taille; i++)
```

```c
        somme += tab[i];
    return somme / taille;  // Bug 3: division entière
}

int main() {
    int valeurs[] = {-5, -2, -8, -1, -3};
    int n = 5;

    printf("Maximum: %d\n", chercher_max(valeurs, n));  // Attendu: -1

    int notes[] = {15, 12, 18, 10, 14};
    printf("Moyenne: %.2f\n", moyenne(notes, 5));  // Attendu: 13.80

    return 0;
}
```

Compiler et déboguer :

```
gcc -g -O0 -Wall buggy.c -o buggy
./buggy  # Observer les résultats incorrects

gdb ./buggy
(gdb) break chercher_max
(gdb) run
(gdb) print taille
(gdb) print tab[0]
(gdb) watch max
(gdb) continue       # Observer les changements de max
(gdb) print i        # Quand i == taille, accès hors limites
(gdb) backtrace
(gdb) break moyenne
(gdb) continue
(gdb) print somme
(gdb) print somme / taille    # Division entière!
(gdb) print (double)somme / taille
(gdb) quit
```

Questions :

1. Pourquoi `chercher_max` retourne 0 pour un tableau de valeurs négatives ?
2. Quelle valeur lit `tab[5]` (accès hors limites) ? Utilisez GDB pour le vérifier.
3. Comment corriger la division entière dans `moyenne` ?
4. Corrigez les trois bugs et vérifiez les résultats attendus.

Livrables

À rendre

1. Un fichier pdf en bonne et due forme contenant les captures, réponses et explications